

FUNCTIONAL DEPENDENCIES, REDUNDANCY AND DATA ANOMALIES IN E-COMMERCE DATABASE

CHAPTER-4

Introduction

An e-commerce database stores and manages information about sellers, customers, products, categories, orders, payments, deliveries, carts, order items, and reviews. A well-designed database should avoid unnecessary data repetition and maintain data consistency. **Functional dependency** explains how one attribute determines another attribute. **Redundancy** refers to the unnecessary repetition of the same data. **Data anomalies** are problems that occur due to poor database design. The three major anomalies are **insertion anomaly, update anomaly, and deletion anomaly**. The following analysis explains the functional dependencies, possible redundancy, and data anomalies for each table in the e-commerce database.

1. CUSTOMER TABLE

Functional Dependency

Customer_ID → Customer_Name, Email, Phone_Number, Address, Password
Customer_ID uniquely determines all customer details.

Redundancy

Customer information may be repeated in the ORDER, CART, and REVIEW tables if the customer's complete details are stored there.

Data Anomalies

- **Insertion Anomaly:** A new customer may not be added without placing an order.
- **Update Anomaly:** Changing a customer's address may require updating many records.
- **Deletion Anomaly:** Deleting a customer's last order could accidentally remove the customer's information.

2. SELLER TABLE

Functional Dependency

$\text{Seller_ID} \rightarrow \text{Seller_Name, Store_Name, Email, Phone_Number}$

Seller_ID uniquely determines all the details of a seller.

Redundancy

Seller information may be repeated if the same seller sells multiple products. For example, the seller's name, email, and phone number should not be stored repeatedly in the PRODUCT table.

Data Anomalies

- **Insertion Anomaly:** A seller cannot be added if the table requires product information.
- **Update Anomaly:** If the seller's email changes, multiple repeated records may need to be updated.
- **Deletion Anomaly:** Deleting the only product of a seller may accidentally delete the seller's information if both are stored in one table.

3. CATEGORY TABLE

Functional Dependency

$\text{Category_ID} \rightarrow \text{Category_Name, Description}$

Category_ID uniquely determines the category name and description.

Redundancy

The same category name and description may be repeated for every product belonging to that category.

Data Anomalies

- **Insertion Anomaly:** A new category cannot be added if a product is required.
- **Update Anomaly:** Changing a category name requires updating many product records.
- **Deletion Anomaly:** Deleting the last product in a category may also remove category information.

4. PRODUCT TABLE

Functional Dependency

Product_ID → **Product_Name, Description, Price, Stock_Quantity, Seller_ID, Category_ID**

Product_ID uniquely determines the details of a product.

Redundancy

Product details may be repeated in the ORDER_ITEM, CART, and REVIEW tables if the product name and price are stored repeatedly.

Data Anomalies

- **Insertion Anomaly:** A product may not be added without a seller or category.
- **Update Anomaly:** Changing product information in multiple tables may result in inconsistent data.
- **Deletion Anomaly:** Deleting a product may accidentally remove related order information if the design is not properly structured.

5. CART TABLE

Functional Dependency

Cart_ID → **Customer_ID, Product_ID, Quantity**

Each Cart_ID identifies a cart record.

Redundancy

Customer and product details may be repeated if their names and other information are stored in the cart table.

Data Anomalies

- **Insertion Anomaly:** A cart item cannot be added without a customer and product.
- **Update Anomaly:** Changes to customer or product details may create inconsistent copies.
- **Deletion Anomaly:** Deleting a cart item should not delete the customer or product information.

6. ORDER TABLE

Functional Dependency

Order_ID → Customer_ID, Order_Date, Order_Status, Total_Amount

Order_ID uniquely identifies an order.

Redundancy

Customer details and product details should not be repeated in the ORDER table.

Data Anomalies

- **Insertion Anomaly:** An order cannot be created without a customer.
- **Update Anomaly:** Repeated customer information may require multiple updates.
- **Deletion Anomaly:** Deleting an order should not delete the customer.

7. ORDER_ITEM TABLE

Functional Dependency

OrderItem_ID → Order_ID, Product_ID, Quantity, Unit_Price

OrderItem_ID uniquely identifies each item in an order.

Redundancy

Product names and customer details should not be repeated for every order item.

Data Anomalies

- **Insertion Anomaly:** An order item cannot be inserted without an order and product.
- **Update Anomaly:** Product details may become inconsistent if repeated.
- **Deletion Anomaly:** Deleting an order item should not delete the product.

8. PAYMENT TABLE

Functional Dependency

Payment_ID → Order_ID, Payment_Method, Payment_Status, Payment_Date, Amount

Payment_ID uniquely identifies a payment.

Redundancy

Order and customer information should not be repeated in the payment table.

Data Anomalies

- **Insertion Anomaly:** A payment cannot be recorded without an order.
- **Update Anomaly:** Repeated order details may cause inconsistent data.
- **Deletion Anomaly:** Deleting a payment should not delete the order.

9.REVIEW TABLE

Functional Dependency

Review_ID → Customer_ID, Product_ID, Rating, Review_Text, Review_Date

Review_ID uniquely identifies a review.

Redundancy

Customer and product details should not be repeated in the review table.

Data Anomalies

- **Insertion Anomaly:** A review cannot be added without a customer and product.
- **Update Anomaly:** Repeated customer or product information may become inconsistent.
- **Deletion Anomaly:** Deleting a review should not delete the customer or product.

10.DELIVERY TABLE

Functional Dependency

Delivery_ID → Order_ID, Delivery_Address, Delivery_Date, Delivery_Status

Delivery_ID uniquely identifies a delivery.

Redundancy

Customer and order details may be unnecessarily repeated in the delivery table.

Data Anomalies

- **Insertion Anomaly:** Delivery cannot be created without an order.
- **Update Anomaly:** Repeated address information may become inconsistent.
- **Deletion Anomaly:** Deleting delivery information should not delete the order or customer.

Conclusion

The e-commerce database design uses separate tables for different entities, which helps reduce data redundancy and maintain data consistency. The primary keys uniquely identify records, while foreign keys establish relationships between tables.

The functional dependencies show that each primary key determines the non-key attributes in its table. By separating customer, seller, product, category, order, payment, delivery, cart, order item, and review information, the database minimizes unnecessary duplication.

Therefore, the database design helps prevent **insertion, update, and deletion anomalies** and provides a more organized, consistent, and efficient structure for managing e-commerce data.